Fuzzing with drchaos

Structured, coverage-guided fuzzing for Nim

Antonis Geralis - NimConf 2022

nim conf 2022

OPENING

The talk in one sentence

Random bytes are useful. Random Nim values are often better.

drchaos lets libFuzzer explore typed data structures instead of making every target hand-parse a byte stream.

> THE MOOD
>
> We still invite chaos. We just give it a type signature.

MAP

What we will cover

1. FUZZING SHAPE

What a fuzzer needs, what it observes, and why coverage changes the game.

2. DRCHAOS MODEL

How a Nim type becomes the fuzz input, including post-processors and custom mutators.

3. TARGETS

Tuples, variants, graphs, and the practical loop from crash to fix.

We will stay practical: what to write, what to constrain, and what a useful crash looks like.

FUZZING

Why fuzz?

Because tests document what we remembered. Fuzzers explore what we forgot.

MODEL

A fuzzer is a feedback loop

GENERATE

Mutate an input corpus into new candidates.

RUN

Execute a small, deterministic target in-process.

LEARN

Keep inputs that discover new coverage or crashes.

> “The interesting part is not randomness. It is the loop between generated input, observed behavior, and future mutations.”
>
> - fuzzing without the fog machine

SIGNALS

What crashes count?

HARD FAILURES

- crashes and panics
- failed assertions
- undefined behavior
- memory leaks and use-after-free

SEMANTIC FAILURES

- roundtrip property broken
- container invariant violated
- parser accepts then cannot print
- API state becomes impossible

```
small target
fast execution
deterministic result
clear failure condition
```

> **RULE OF THUMB**
>
> If a target needs a network, a clock, or a meeting to explain, it is probably too large.

PROBLEM

The byte-stream tax

CLASSIC SHAPE

```
proc LLVMFuzzerTestOneInput(
    data: ptr uint8;
    size: csize_t
): cint =
  let input = parse(data, size)
  exercise(input)
```

WHERE EFFORT GOES

- splitting bytes into fields
- building nested dynamic values
- keeping variants internally valid
- reaching deep comparisons

> **DRCHAOS ANGLE**
>
> Make the target type explicit, then mutate structured values directly.

DRCHAOS

Enter drchaos

LibFuzzer plus sanitizers, with a Nim-aware structured mutator in the middle.

STACK

What is drchaos?

LLVM LIBFUZZER

The engine: coverage, corpus, mutation scheduling, crash minimization.

SANITIZERS

The tripwires: AddressSanitizer and UndefinedBehaviorSanitizer.

NIM MUTATORS

The translator: bytes become strings, tuples, objects, enums, refs, arrays, sets, and seqs.

fuzzing/split-inputs.md at master · google/fuzzing — Mozilla Firefox

fuzzing/split-inputs.md

https://github.com/google/fuzzing/blob/master/docs 120%

236 lines (188 sloc) 11.6 KB Raw Blame

Type-length-value

A custom Type-length-value, or TLV, may sound like a good solution. However, we typically **do not recommend using a custom TLV** to split your fuzzer-generated input for the following reasons:

- This is more test-only code for you to maintain, and easy to get wrong
- Typical mutations performed by fuzzing engines, such as inserting a byte, will break the TLV structure too often, making fuzzing less efficient

However, a TLV input combined with a custom mutator might be a good option. See Structure-Aware Fuzzing.

Protobufs

Yet another option is to use one of the general-purpose serialization formats, such as Protobufs, in combination with a custom mutator. See Structure-Aware Fuzzing.

API

The API surface is tiny

```
import drchaos

func fuzzTarget(data: MyInput) =
  # Use data. Check invariants.
  # Do not mutate data.

defaultMutator(fuzzTarget)
```

THE TARGET SIGNATURE IS THE SCHEMA

- `MyInput` drives serialization
- equality support affects several generated types
- `default` can define valid refs
- `postProcess` repairs dependencies

NIM IDENTITY

> The target stays ordinary Nim. The strange part lives in the mutator framework.

TUPLE

A deliberately simple target

```
import drchaos

proc fuzzMe(s: string, a, b, c: int32) =
  if a == 0xdeadc0de'i32 and
      b == 0x11111111'i32 and
      c == 0x22222222'i32:
    if s.len == 100:
      doAssert false

func fuzzTarget(data: (string, int32, int32, int32)) =
  let (s, a, b, c) = data
  fuzzMe(s, a, b, c)

defaultMutator(fuzzTarget)
```

WHY THIS WORKS

The tuple gives the fuzzer handles for each field. Value profiling helps it cross exact comparisons.

WHY THIS IS ONLY A WARM-UP

The payoff arrives when input is nested, variant-heavy, or graph-shaped.

SHIFT

Structured inputs change the target

BEFORE

Parse arbitrary bytes, reject most cases, then maybe reach the API.

AFTER

Generate values that resemble the domain, then spend cycles on behavior.

ENUMS

finite choices

VARIANTS

shape follows tag

SEQS

bounded growth

REFS

defaults avoid nil traps

VALIDITY

Post-processors keep relations valid

```
proc postProcess[T](x: var seq[Node[T]]; r: var Rand) =
  for n in x.mitems:
    for i in countdown(n.edges.high, 0):
      if n.edges[i].int >= x.len:
        del(n.edges, i)
```

WHAT IT SAYS

Edges may only point to nodes that exist.

WHY NOT ENCODE EVERYTHING IN MUTATE?

Mutators explore. Post-processors restore cross-field invariants visibly.

TARGETS

Graph target

The demo target is a small graph library: nodes, edges, BFS, and invariants.

DEMO

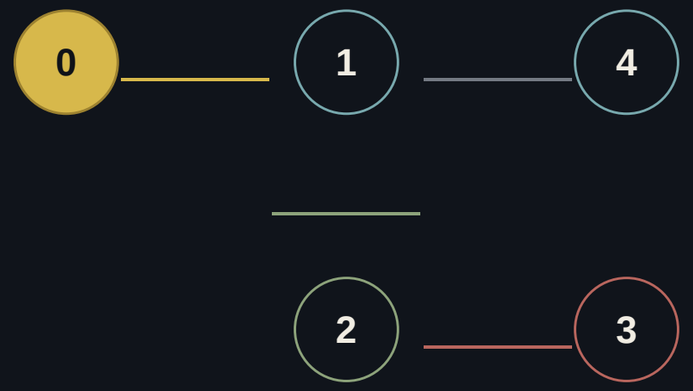
The graph gives chaos some topology

DOMAIN

- graph as adjacency list
- node indices as a distinct type
- bounded node and edge counts
- BFS as observable behavior

TARGET INVARIANT

Traversal should not crash, and every visited node should be reachable from the source.



A small adjacency list is enough: structure first, behavior second.

MUTATORS

Constrain the search space

```
const
  MaxNodes = 8
  MaxEdges = 2

proc mutate(
    value: var NodeIdx;
    sizeIncreaseHint: int;
    enforceChanges: bool;
    r: var Rand
) =
  repeatMutate(mutateEnum(value.int, MaxNodes, r).NodeIdx)
```

THIS IS NOT CHEATING

Bounds make the state space productive. The fuzzer still explores combinations; it stops inventing node 90 in a graph with 3 nodes.

USE DISTINCT TYPES

A distinct `NodeIdx` gets a domain-specific mutator without changing the production representation outside fuzz builds.

INVARIANTS

Target shape for BFS

```
func fuzzTarget(x: Graph[int8]) =
  when defined(dumpFuzzInput):
    debugEcho(x)

  if x.len > 0:
    discard x.breadthFirstSearch(source = 0.NodeIdx)
```

START SMALL

A crash-only target is still valuable when the generated input has realistic shape.

THEN ADD PROPERTIES

Remove an edge, preserve node count. Traverse from a source, ensure each result is reachable.

CONFIG

The compile line is part of the tool

```
--cc:clang
-d:useMalloc
-t:"-fsanitize=fuzzer,address,undefined"
-l:"-fsanitize=fuzzer,address,undefined"
-d:nosignalhandler
--nomain:on
--mm:arc
-g
```

CLANG

libFuzzer integration

SANITIZERS

memory and UB visibility

ARC/ORC

better fit for variants and modern Nim

PRACTICE

Crash workflow

FIND

Run fast with sanitizers and a minimal target.

REPRODUCE

Recompile with debug info; pass the crash artifact back in.

EXPLAIN

Turn the crash into a regression test or a narrower invariant.

PRACTICAL NOTE

Use `debugEcho(x)` behind a flag when you need the generated Nim value. Keep hot fuzz loops quiet.

CHECKLIST

What makes a good target?

DO

- keep it deterministic
- keep it fast enough for millions of runs
- check a clear property
- prefer small, typed inputs
- bound uninteresting growth

AVOID

- printing every case
- mutating the input value in the target
- depending on external services
- swallowing crashes into generic errors
- testing five subsystems at once

FIT

Where drchaos is sharpest

PARSERS AND PRINTERS

Roundtrip properties, AST variants, escaping rules, boundary values.

CONTAINERS

Insert/delete invariants, iterator safety, edge cases around empty data.

STATEFUL APIS

Command sequences, model checks, generated histories, offline simulations.

> “The fuzzer is not a proof. It is a persistent reviewer with unusual taste in inputs.”

BOUNDARIES

Known limits

CURRENTLY AWKWARD

- polymorphic serialization
- reference cycles
- raw pointers without user serializers
- object variants outside modern memory management

DESIGN RESPONSE

- wrap raw pieces with explicit serializers
- define `default` for non-nil refs
- use distinct types to narrow fields
- keep generated structure honest with `postProcess`

FUTURE

Future directions

Richer mutators, stateful APIs, and model-heavy simulation work.

NEXT

Future directions

The next step is less random noise, more domain signal.

Richer mutator architecture, generated command streams, and offline simulation work all push the same direction: make exploration resemble the system under test.

CLOSE

Questions?

Preferably the kind that produce smaller reproducers.

PROJECT SUPPORT

zah and Status funded drchaos. disruptek sponsored the work. Nim community testing made the rough edges visible.

THANK YOU

Antonis Geralis - NimConf 2022